

ME315 - Lab 03

Author

Luiza Tortorelli

```
install.packages(c("readr", "dplyr", "tidyr"))
```

```
Installing packages into '/cloud/lib/x86_64-pc-linux-gnu-library/4.6'
(as 'lib' is unspecified)
```

```
library(readr)
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':
```

```
filter, lag
```

```
The following objects are masked from 'package:base':
```

```
intersect, setdiff, setequal, union
```

```
library(tidyr)
```

Atrasos de Voos

1. Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

- Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;

```
airlines <- read_csv("airlines.csv")
```

```
Rows: 14 Columns: 2
```

```
— Column specification —————
```

```
Delimiter: ","
```

```
chr (2): IATA_CODE, AIRLINE
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
airports <- read_csv("airports.csv",
                    col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE))
```

```
Rows: 322 Columns: 5
```

```
— Column specification —————
```

```
Delimiter: ","
```

```
chr (3): IATA_CODE, CITY, STATE
dbl (2): LATITUDE, LONGITUDE
```

• Use ``spec()`` to retrieve the full column specification for this data.
 • Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

- Para o arquivo de vôos:
 - a. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`;
 - b. Leia apenas 1 milhão de linhas por vez;
 - c. Remova vôos em que o aeroporto de destino comece com a letra 1;
 - d. Remova registros em que existam pelo menos uma coluna faltante;
 - e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
 - f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
processar_voos <- function(chunk, pos) {
  chunk |>
    filter(!startsWith(DESTINATION_AIRPORT, "1")) |>
    drop_na() |>
    group_by(DESTINATION_AIRPORT) |>
    summarise(
      soma_atraso = sum(ARRIVAL_DELAY),
      qtd_voos = n(),
      .groups = 'drop'
    )
}
```

```
estatisticas_chunks <- read_csv_chunked(
  "flights.csv",
  callback = DataFrameCallback$new(processar_voos),
  chunk_size = 1000000,
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)
```

```
flights <- estatisticas_chunks |>
  group_by(DESTINATION_AIRPORT) |>
  summarise(
    soma_total = sum(soma_atraso),
    voos_totais = sum(qtd_voos),
    .groups = 'drop'
  ) |>
  mutate(ATRASO_MEDIO = soma_total / voos_totais) |>
  select(DESTINATION_AIRPORT, ATRASO_MEDIO)
```

□

2. Selecione a operação apropriada join para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

- a. Identifique a coluna que é a chave na tabela `flights`;

Resposta: A chave na tabela `flights` é `DESTINATION_AIRPORT`.

- b. Identifique a coluna que é a chave na tabela `airports`;

Resposta: A chave na tabela `airports` é `IATA_CODE`.

- c. Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?

Resposta: Não há nenhum aeroporto listado em flights que não esteja listado em airports.

```
aeroportos_ausentes <- anti_join(flights, airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
print(aeroportos_ausentes$DESTINATION_AIRPORT)
```

—

```
character(0)
```

d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves; *Resposta:* O comando que combina as tabelas é o `left_join`

```
left_join(flights, airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
```

—

```
# A tibble: 322 × 6
  DESTINATION_AIRPORT ATRASO_MEDIO CITY          STATE LATITUDE LONGITUDE
  <chr>              <dbl> <chr>          <chr>    <dbl>    <dbl>
1 ABE                5.80 Allentown    PA        40.7     -75.4
2 ABI                4.08 Abilene      TX        32.4     -99.7
3 ABQ                5.61 Albuquerque NM        35.0    -107.
4 ABR               -3.39 Aberdeen    SD        45.4     -98.4
5 ABY                8.68 Albany      GA        31.5     -84.2
6 ACK                4.91 Nantucket    MA        41.3     -70.1
7 ACT               10.6 Waco         TX        31.6     -97.2
8 ACV                9.68 Arcata/Eureka CA        41.0    -124.
9 ACY               11.8 Atlantic City NJ        39.5     -74.6
10 ADK               -3.19 Adak       AK        51.9    -177.
# i 312 more rows
```

e. Armazene a tabela resultante no objeto `flights`.

```
flights <- left_join(flights, airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
```

—

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
aeroportos_por_estado <- airports |>
  group_by(STATE) |>
  summarise(total_aeroportos = n()) |>
  arrange(desc(total_aeroportos))

print(aeroportos_por_estado)
```

—

```
# A tibble: 54 × 2
  STATE total_aeroportos
  <chr>          <int>
1 TX              24
2 CA              22
3 AK              19
4 FL              17
5 MI              15
6 NY              14
7 CO              10
8 MN               8
9 MT               8
10 NC              8
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

a. Carregue o pacote leaflet;

```
install.packages("leaflet")
```

—

Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.6'
(as 'lib' is unspecified)

```
library(leaflet)
```

—

b. Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico;

c. Armazene o grafico em b) numa variável chamada this;

```
flights_mapa <- flights |>
  drop_na(LATITUDE, LONGITUDE)

this <- leaflet(data = flights_mapa) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~paste("Aeroporto:", DESTINATION_AIRPORT,
                   "<br>Atraso Médio:", round(ATRASO_MEDIO, 2), "minutos")
  )

this
```

—

Horário

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```



```
[1] "2026-08-31 23:21:43 -03"
```